Disciplina: CCP230 — Desenvolvimento de Algoritmos

Instituição: Centro Universitário FEI

Docente: Prof. Dr. André Luiz Perin

Aluno: Pedro Megiolaro Braga **RA:** 52.126.030-7

1. INTRODUÇÃO E OBJETIVOS

O presente projeto consiste no desenvolvimento de uma aplicação em linguagem C para simular o funcionamento de uma **Exchange de Criptomoedas** no próprio console. A plataforma permite a gestão de uma carteira de investimentos digital, possibilitando a consulta de saldos, extratos detalhados com marcas temporais, depósitos e saques em moeda corrente , compra e venda de criptoativos e atualização de cotações de mercado.

O sistema **individualmente**, focando no perfil do usuário investidor e na persistência completa dos dados através de arquivos de texto(.txt), fazendo com que nenhuma informação seja perdida entre execuções do programa.

2. ARQUITETURA DO SISTEMA E ESTRUTURAS DE DADOS

A aplicação foi estruturada de forma modular, dividindo as responsabilidades do sistema em dados de investimento, cotações de mercado e manipulação de ficheiros.

2.1. Estruturas de Dados

Para representar a carteira do utilizador e as cotações de mercado, foram definidas duas estruturas fundamentais:

- typedef struct {
- char nome[100];
- char cpf[15];
- char senha[10];
- double reais;
- double bitcoin;
- double ethereum;
- double ripple;
- } Investidor;

Investidor: Armazena as credenciais de acesso (CPF e senha de 6 dígitos), o nome do utilizador e as quantidades de saldo em Reais (BRL), Bitcoin (BTC), Ethereum (ETH) e Ripple (XRP).

- typedef struct {
- double bitcoin;
- double ethereum;
- double ripple;
- } Cotacoes;

- **Cotações**: Armazena os valores de mercado atualizados das três criptomoedas suportadas.

3. PERSISTÊNCIA DE DADOS EM ARQUIVOS(.txt)

Conforme os requisitos, todos os dados da plataforma são persistidos no disco local:

1. **investidor.txt**: Guarda os dados cadastrais e o estado atual da carteira de investimentos.
2. **cotacoes.txt**: Regista os valores atuais de cada criptomoeda.
3. **extrato_[CPF].txt**: Ficheiro de registo histórico (*log*) individual criado dinamicamente com base no CPF do investidor, onde cada operação de depósito, saque, compra ou venda é gravada de forma incremental.

4. REGRAS DE NEGÓCIO E FUNCIONALIDADES

4.1. Autenticação (Login)

- O usuário insere o CPF (sem pontos e traços) e a senha de 6 dígitos.
- O sistema valida as credenciais com as informações lidas do arquivo **investidor.txt.** Em caso de sucesso, dá acesso ao menu de opções.

4.2. Consulta de Saldo e Extrato

- **Saldo**: Exibe o valor em Reais e o número de unidades acumuladas em BTC, ETH e XRP.
- **Extrato**: Mostra o histórico de transações contendo data, hora, tipo de operação (+ ou -), valor, taxas cobradas e saldos resultantes.

4.3. Operações Financeiras em Reais (BRL)

- **Depósito**: Aumenta o saldo em Reais do investidor e regista a operação no extrato.
- **Saque**: Exige a confirmação de senha e Diminui o saldo em Reais, assegurando a regra de que o saldo não pode ficar negativo.

4.4. Compra e Venda de Criptomoedas

As operações aplicam a tabela de taxas obrigatórias e garantem a não-negatividade dos saldos em carteira:

Criptomoeda	**Taxa de Compra Projeto - Exchange de Criptomoedas.pdf**	**Taxa de Venda Projeto - Exchange de Criptomoedas.pdf**
Bitcoin (BTC)	2%	3%
Ethereum (ETH)	1%	2%

Ripple (XRP)	1%	1%

Fórmula de Compra: Taxa = ValoremReais x TaxadeCompra

ValorLíquido = ValoremReais -Taxa

QuantidadeAdquirida=ValorLíquido / CotaçãoAtual

Fórmula de Venda: ValorBruto = Qtd.Cripto x CotaçãoAtual

Taxa = ValorBrutox TaxadeVenda

ValorCreditadoemReais = ValorBruto - Taxa

4.5. Atualização de Cotações

- As cotações sofrem oscilação aleatória dentro do intervalo fechado entre **-5% e +5%** do valor anterior.
- A variação é calculada utilizando a função rand() com a semente configurada por srand(time(NULL)).

5. CONSIDERAÇÕES DE IMPLEMENTAÇÃO E TESTES

O desenvolvimento priorizou o uso de construções claras e padrão da Linguagem C (stdio.h, stdlib.h, string.h, time.h), evitando bibliotecas externas.

- **Tratamento de Validações**: O código impede entradas inválidas (valores negativos ou nulos) e valida a disponibilidade de fundos antes de efetivar saques ou vendas.
- **Persistência**: Testes efetuados confirmaram que a abertura e o fecho seguidos da aplicação mantêm a integridade dos saldos e do histórico de transações.

6. CONCLUSÃO

A solução desenvolvida atende a todos os requisitos funcionais e não-funcionais estabelecidos no enunciado do projeto individual. A estrutura em C provou ser eficiente no manuseamento de arquivos de texto para persistência e no controle de regras de negócio financeiras.